TECHNICAL WORKFLOW GUIDE

Health Insurance Management System (HIMS)

— Koushik Gaddam

Document Type	Business Workflow & Technical Guide
Version	1.0 — Final
Domain	InsurTech / Health Insurance
Tech Stack	.NET 8 · Angular 17 · SQL Server · JWT · QuestPDF
Date	12 March 2026
Author	Koushik Gaddam

© 2025 Koushik Gaddam. All Rights Reserved.

Table of Contents

1. System Overview & Core Constants 3
2. Workflow 1 — User Registration & KYC Onboarding 5
2A. Path A: Customer Self-Registration 5
2B. Path B: Agent-Created Customer 6
3. Workflow 2 — Internal Staff Provisioning 7
4. Workflow 3 — Premium Quotation Engine 8
5. Workflow 4 — Policy Activation & Document Gate 10
6. Workflow 5 — Mid-Term Policy Changes & Endorsements 13
7. Workflow 6 — Claim Adjudication & Coverage Deduction 15
8. Workflow 7 — Background Automation Service 18
9. Database, Audit Trail & Soft Delete Standards 19

1. System Overview & Core Constants

This section defines the immutable technical constants that govern all financial calculations, state transitions, and commission logic throughout the HIMS Portal. Every workflow in this document operates within these boundaries.

1.1 Technology Stack

Component	Details
Frontend	Angular 17+ — Standalone Components, Signals for state management, Lazy-loaded feature modules
Backend	.NET 8 Web API — Clean Architecture (Domain / Application / Infrastructure / API layers)
Database	Microsoft SQL Server — Entity Framework Core 8 (Code-First migrations, decimal(18,2) precision)
PDF Engine	QuestPDF — Invoice PDFs and Claim Settlement Documents
Auth	JWT (JSON Web Tokens) with BCrypt password hashing — role-bound claims (Customer, Agent, ClaimsOfficer, Admin)
Background	PolicyExpiryWorker / InvoiceWorker (IHostedService) — runs every 24 hours

1.2 Financial & Timeline Constants

⚠ These constants are sourced directly from BusinessRules.cs and must not be overridden by any controller or service logic.

Constant	Value / Rule
Age Premium Increment	Base Premium increases by +10% for every 10 years of the prospect's age: Multiplier = 1 + ((Age ÷ 10) × 0.10)
Coverage Formula	CoverageAmount = MonthlyPremium × 120 × TierMultiplier
Annual Discount	Not applicable in HIMS v1.0 — monthly billing only
Agent Commission Rate	5% flat on every MonthlyPremium amount upon policy activation
KYC Strike Rule	After 3 document rejections by an Agent, the policy application is permanently blocked
Claim Deduction	Approved ClaimAmount is permanently and atomically deducted from the policy's CoverageAmount
Password Length	10 high-entropy characters for auto-generated temporary credentials (Path B onboarding)
Document Gate	A policy cannot proceed to the Payment stage if ANY document is in Pending or Rejected status

1.3 Tier Multipliers (PolicyService.cs)

ℹ Multipliers are applied to BOTH the monthly premium and the coverage amount. Higher tiers increase both cost and coverage proportionally.

Plan Tier	Display Name	Premium Multiplier	Coverage Multiplier	Target Segment
Silver	Standard Care	1.0×	120 × Premium × 1.0	Budget-conscious individuals
Gold	Premium Care	1.2×	120 × Premium × 1.2	Families seeking mid-range coverage
Platinum	Elite Care	1.5×	120 × Premium × 1.5	High-value / comprehensive coverage

1.4 Premium Calculation Formula

The HIMS premium engine applies two sequential adjustments to arrive at the final monthly premium:

```
Monthly Premium = Base Premium × (1 + (Age ÷ 10) × 0.10) × TierMultiplier
Coverage Amount = Monthly Premium × 120 × TierMultiplier
```

Age	Silver (1.0×) | Base ₹500	Gold (1.2×) | Base ₹500	Platinum (1.5×) | Base ₹500
25 years	₹500 × 1.25 × 1.0 = ₹625	₹500 × 1.25 × 1.2 = ₹750	₹500 × 1.25 × 1.5 = ₹937.50
35 years	₹500 × 1.35 × 1.0 = ₹675	₹500 × 1.35 × 1.2 = ₹810	₹500 × 1.35 × 1.5 = ₹1,012.50
45 years	₹500 × 1.45 × 1.0 = ₹725	₹500 × 1.45 × 1.2 = ₹870	₹500 × 1.45 × 1.5 = ₹1,087.50
55 years	₹500 × 1.55 × 1.0 = ₹775	₹500 × 1.55 × 1.2 = ₹930	₹500 × 1.55 × 1.5 = ₹1,162.50

1.5 Agent Commission Logic

ℹ Commission is triggered ONLY on successful policy activation (payment confirmed) — never on policy creation, quote generation, or document upload. Every calculation creates an immutable AuditLog entry.

```
Agent Commission = MonthlyPremium × 0.05
```

Example: A customer activates a Gold policy at age 35 with Base ₹500 → Monthly Premium = ₹810 → Agent Commission = ₹810 × 5% = ₹40.50 credited to the Agent on activation.

2. Workflow 1 — User Registration & KYC Onboarding

Field	Detail
Goal	Onboard a new individual customer through either self-service or agent-assisted registration. Both paths converge at the Agent KYC document verification stage.
Actors	Customer, Agent, Admin, System
Entry Points	Public signup page (Path A) OR Agent dashboard (Path B)
Final State	Customer KYC approved, assigned to Agent, policy-ready
Key Guard	ALL KYC documents must be Agent-verified before any policy can proceed to the payment stage

2A. Path A — Customer Self-Registration

REGISTERED → DRAFT → PENDING → KYC UPLOADED → VERIFIED → POLICY-READY

1. Customer navigates to the public signup page and registers with their email address and a chosen password. A User record is created in the database with Role = 'Customer' and Status = 'Pending'.
2. Customer completes their personal profile: Full Name, Date of Birth, Phone Number, and residential address.
3. KYC Upload: Customer uploads mandatory identity documents — Aadhar Card, PAN Card, and a passport-sized photograph. Document records are created in the DocumentVault table with Status = 'Pending'. Policy application status transitions to 'KYCUploaded'.
4. System triggers a notification to the assigned Agent's dashboard queue. The Agent's inbox displays the new pending verification.
5. Agent Review: Agent opens each document in the DocumentVault viewer and individually sets the status to 'Verified' or 'Rejected'.
6. Strike Rule: If an Agent rejects a document, the Customer is notified and may re-upload a corrected version. After 3 rejections (RejectionCount = 3 on the PremiumQuote record), the application is PERMANENTLY BLOCKED. No further submissions are accepted.
7. On full KYC approval (all documents Verified), the Document Gate is cleared. The policy application transitions to 'Approved' status. Customer receives notification that they may now proceed to payment.
8. Customer is directed to the payment portal. Payment confirmation triggers immediate policy activation and PDF invoice generation.

2B. Path B — Agent-Created Customer

AGENT CREATES → PENDING → KYC UPLOADED → VERIFIED → CREDS SENT → FORCE RESET

9. Agent creates a Customer account directly from the Agent Dashboard — enters the customer's personal details on their behalf.
10. Agent uploads KYC documents on the customer's behalf. The customer is permanently linked to this Agent from creation — no re-assignment is possible.
11. Account enters the verification queue in PENDING status. The same Agent (or an Admin) reviews and approves the KYC documents.
12. On Rejection: The creating Agent is notified to re-upload corrected documents. The same 3-strike limit applies.

13. On Approval: System generates a secure 10-character temporary password using high-entropy characters (uppercase, lowercase, digits, special characters) and sends it to the Customer's registered email address.
14. Forced Password Reset: Customer logs in with temporary credentials and is immediately redirected to a mandatory password change screen. The requirePasswordChange flag is set to true on the User record. No dashboard access is granted until this step is completed.
15. Customer gains full dashboard access. Account is policy-ready and permanently associated with the creating Agent.

ℹ Key Difference: In Path A, the customer self-registers and may be assigned to any Agent. In Path B, the Agent-to-Customer bond is permanent from the moment of account creation.

3. Workflow 2 — Internal Staff Provisioning

Field	Detail
Goal	Securely add Agents and Claims Officers with enforced first-login credential hygiene.
Actors	Admin, System, New Staff Member (Agent or ClaimsOfficer)
Trigger	Admin initiates from Admin Dashboard → Staff Management panel
Output	Active staff account with role-bound JWT claim + forced password reset on first login
Roles Created	Agent, ClaimsOfficer

ADMIN INITIATES → ACCOUNT CREATED → EMAIL SENT → FIRST LOGIN → ACTIVE

16. Admin Input: Admin enters the new staff member's Full Name, Email Address, and selects their Role (Agent or ClaimsOfficer) from the Staff Management panel.
17. Credential Generation: System generates a secure 10-character temporary password using high-entropy characters. This password is single-use and cannot be reused after the first login.
18. Account Creation: A User record is created with the specified Role encoded as a JWT claim. The MustChangePassword flag is set to true. Account status is Active from creation — the staff member can log in immediately.
19. Email Dispatch: System sends an automated welcome email to the new staff member containing their login email address and the temporary password.
20. First Login Guard: On initial login, the ASP.NET Core middleware stack detects MustChangePassword = true in the JWT claims and strictly intercepts all routing. The staff member is redirected to the password update screen. No application features — dashboards, queues, or actions — are accessible until the password is changed.
21. Staff member sets a new personal password. BCrypt hashes the new password and writes it to User.PasswordHash. The MustChangePassword flag is cleared. Full role-appropriate dashboard access is granted.

⚠ Security Note: The MustChangePassword flag cannot be bypassed via any API endpoint. It is enforced server-side at the middleware layer — client-side manipulation has no effect.

3.1 Role Capabilities Reference

Role	Accessible Modules	Key Permissions
Agent	Customer Queue, KYC Verification, Policy Dashboard, Commission History	Verify/Reject KYC docs; Approve/Reject endorsements; View assigned customers
ClaimsOfficer	Claims Queue, Document Review, Settlement Panel	Take/Release claim locks; Approve/Reject claims; Generate PDF settlements
Admin	All modules including Staff Management, Audit Logs, Plan Templates	Create staff accounts; Create plan templates; Access full audit trail; Force actions
Customer	Personal Dashboard, Quote Engine, Claims Submission	Calculate quotes; Upload KYC; Make payments; Submit claims; View policy status

4. Workflow 3 — Premium Quotation Engine

Field	Detail
Goal	Enable prospects and registered customers to calculate accurate, real-time insurance premiums before committing to a policy application.
Actors	Customer (Anonymous or Authenticated), System (PremiumService.cs)
Trigger	User lands on the Quote Engine — accessible without authentication
Service Type	Transient-scoped — a fresh PremiumService instance is created per request, preventing state contamination
Output	Calculated MonthlyPremium, CoverageAmount, and a persisted PremiumQuote record (if authenticated)

4.1 Quote Calculation Flow

PLAN SELECTED → AGE ENTERED → ENGINE RUNS → RESULT DISPLAYED → QUOTE SAVED

22. User navigates to the public Quote Engine (GET /api/quote/calculate). No authentication is required at this stage — anonymous calculation is fully supported.
23. User selects a Plan Tier: Silver, Gold, or Platinum. The corresponding TierMultiplier is loaded from the admin-configured Plan Templates table.
24. User enters their age. The system validates: Age must be 18 or older. If age < 18, the request is rejected with a validation error before any calculation occurs.
25. PremiumService.cs executes the calculation: Step 1: AgeMultiplier = 1 + ((Age ÷ 10) × 0.10) Step 2: MonthlyPremium = BasePremium × AgeMultiplier × TierMultiplier Step 3: CoverageAmount = MonthlyPremium × 120 × TierMultiplier
26. Result is returned to the frontend. The Quote Engine displays the MonthlyPremium, CoverageAmount, and a breakdown of the calculation for full transparency.
27. If the user is authenticated, the system persists the result as a PremiumQuote record in the database with IsConvertedToPolicy = 0 (PendingReview) and a unique QuoteReference (QUO-XXXX). This becomes the foundation for the policy application.
28. If the user is anonymous, no record is created. They are prompted to register or log in to save the quote and proceed to the policy application.

4.2 Complete Worked Examples

Scenario	Input	Calculation Steps	Result
Young Silver	Age 25, Silver, Base ₹500	1+(25÷10×0.10)=1.25 → ₹500×1.25×1.0	₹625/mo, ₹75,000 cover
Mid Gold	Age 35, Gold, Base ₹500	1+(35÷10×0.10)=1.35 → ₹500×1.35×1.2	₹810/mo, ₹1,16,640 cover
Senior Platinum	Age 55, Platinum, Base ₹500	1+(55÷10×0.10)=1.55 → ₹500×1.55×1.5	₹1,162.50/mo, ₹2,09,250 cover
Max Age Silver	Age 99, Silver, Base ₹500	1+(99÷10×0.10)=1.99 → ₹500×1.99×1.0	₹995/mo, ₹1,19,400 cover

4.3 Plan Template Data Model

Plan Templates are created exclusively by Admins through the POST /api/policy/create endpoint. They serve as the source-of-truth for base premiums and are referenced by the PremiumService at calculation time.

Field	Type	Description
Id	INT, PK	Auto-incremented identifier
PlanName	NVARCHAR(100)	Silver / Gold / Platinum
BasePremium	DECIMAL(18,2)	Admin-configured base monthly premium
TierMultiplier	DECIMAL(5,2)	1.0 / 1.2 / 1.5 — applied to premium and coverage
IsPlanTemplate	BIT, DEFAULT 1	Discriminator flag — distinguishes templates from active policies
IsActive	BIT, DEFAULT 1	Soft delete flag — inactive templates are hidden from the quote engine

⚠ All monetary fields (BasePremium, MonthlyPremium, CoverageAmount, ClaimAmount) are stored as DECIMAL(18,2) in SQL Server. This is enforced via EF Core Fluent API configuration and prevents any floating-point rounding errors in financial calculations.

5. Workflow 4 — Policy Activation & The Document Gate

Field	Detail
Goal	Transform a saved PremiumQuote into a live, active insurance policy through the KYC verification gate and payment confirmation.
Actors	Customer, Agent, System (PaymentController.cs, InvoiceService.cs)
Trigger	Customer submits a policy application from a saved PremiumQuote
The Document Gate	The single most critical business rule — payment is BLOCKED unless ALL documents are in 'Verified' status
Output	Active policy with IsPaid = true, CoverageAmount live, PDF invoice stored in DocumentVault

5.1 Full Policy State Machine

State	IsConvertedToPolicy	IsPaid	Description
PendingReview	0	false	Application submitted; documents uploaded; awaiting Agent review
Approved	1	false	All documents Verified by Agent; Document Gate cleared; awaiting payment
Active	1	true	Payment confirmed; policy live; CoverageAmount available for claims
Rejected	2	false	Terminal state — Agent rejected documents 3 times; no further appeal
Inactive	1	true	Policy expired or manually deactivated; claims and payments blocked

5.2 Step-by-Step Policy Activation Flow

QUOTE SAVED → DOCS UPLOADED → AGENT REVIEW → DOCUMENT GATE → PAYMENT → ACTIVE

29. Customer selects a saved PremiumQuote from their dashboard and initiates a policy application by clicking 'Apply'. The PremiumQuote.IsConvertedToPolicy is confirmed as 0 (PendingReview).
30. Customer uploads mandatory KYC documents. HIMS requires at minimum: Aadhar Card (DocumentType = 'Aadhar') and a Government-issued Photo ID (DocumentType = 'PhotoID'). Each document is stored in the DocumentVault table with Status = 'Pending' and RelatedEntityType = 'Policy'.
31. System triggers a notification to the assigned Agent. The application appears in the Agent's verification queue with the customer's name, plan tier, and uploaded document count.
32. Agent opens each document in the DocumentVault viewer. For each document, the Agent sets the Status to either 'Verified' or 'Rejected'. The Agent may also leave a review note.

33. If a document is Rejected, the Customer receives an email notification with the Agent's rejection reason. The Customer re-uploads the corrected document, resetting that document's status to 'Pending' and triggering re-notification to the Agent. The RejectionCount on the PremiumQuote is incremented.
34. Strike Enforcement: If RejectionCount reaches 3, the system automatically sets IsConvertedToPolicy = 2 (Rejected) and permanently blocks the application. No further uploads or appeals are accepted on this quote. The Customer must start a new quote.
35. Document Gate Check (StaffController.VerifyPolicy): When the Agent attempts to approve the policy, the system executes the following logic: • Query: SELECT COUNT(*) FROM DocumentVault WHERE RelatedEntityId = quoteId • IF any document.Status = 'Rejected' → ABORT. Auto-set application to Rejected. • IF any document.Status = 'Pending' → ABORT. Return HTTP 400: 'All documents must be reviewed.' • IF all documents.Status = 'Verified' → PROCEED to approval.
36. On successful Document Gate passage: PremiumQuote.IsConvertedToPolicy = 1. Customer receives an approval notification and a payment prompt is activated on their dashboard.
37. Customer completes payment via the payment portal. The PaymentController.ProcessPayment() method executes: • Validates quote approval status (must be IsConvertedToPolicy = 1) • Sets PremiumQuote.IsPaid = true • Calls InvoiceService.GenerateInvoice() to produce a PDF invoice via QuestPDF • Stores the invoice PDF in DocumentVault (DocumentType = 'Invoice', Status = 'Verified') • Creates an AuditLog entry: ActionType = 'PolicyActivation'
38. Policy is now ACTIVE. The CoverageAmount is live on the customer's dashboard. The Agent's commission (5% of MonthlyPremium) is calculated and recorded in AgentCommissionLog.

5.3 The Document Gate — Detailed Logic

⚠ The Document Gate is the most critical business rule in the HIMS policy workflow. It is enforced server-side and cannot be bypassed by any client-side interaction or API parameter.

Gate Check	Query Logic	Outcome if Failed	Outcome if Passed
All Docs Reviewed?	COUNT(docs WHERE Status='Pending') = 0	HTTP 400: Pending documents exist	Continue to next check
Any Docs Rejected?	COUNT(docs WHERE Status='Rejected') > 0	Auto-Reject application; notify customer	Continue to approval
All Docs Verified?	COUNT(docs WHERE Status='Verified') = ALL docs	N/A — handled above	IsConvertedToPolicy = 1; Payment unlocked

5.4 Invoice Generation (InvoiceService.cs)

Upon payment confirmation, QuestPDF generates a branded insurance invoice. The invoice is stored as binary data in the DocumentVault and made immediately downloadable from the customer's dashboard.

Invoice Section	Content	Data Source
Header	Transaction Reference, Company Name (Health Insurance Inc.), Payment Date	PaymentController params
Billed To	Customer Full Name, Email address	User.FullName, User.Email

Invoice Section	Content	Data Source
Policy Details	PlanName, TierName, CoverageAmount activated	PremiumQuote entity
Transaction Table	Description, Period (1 Month), MonthlyPremium rate, Total Paid	PremiumQuote.CalculatedMonthlyPremium
Footer	Auto page number, compliance disclaimer, issue date	System-generated

6. Workflow 5 — Mid-Term Policy Changes & Endorsements

Field	Detail
Goal	Handle policyholder profile changes mid-contract — plan upgrades and coverage adjustments — with accurate financial recalculation.
Actors	Customer, Agent, Admin, System
Trigger	Customer requests a change from their active policy dashboard
Endorsement Types	PlanUpgrade, CoverageAdjustment, ContactUpdate, BeneficiaryChange
Output	Updated policy record, recalculated premium if applicable, AuditLog entry

6.1 Endorsement Submission & Review Flow

SUBMITTED → PENDING → AGENT REVIEW → APPROVED / REJECTED

39. Customer navigates to their active policy on the dashboard and selects 'Request Change'. A list of permitted endorsement types is displayed based on the current policy state.
40. Customer selects endorsement type and provides supporting details. For a PlanUpgrade request, the customer selects the new tier (e.g., Silver → Gold) and provides a justification. The request is serialized as JSON and stored on the PolicyEndorsement record.
41. Endorsement Status = PENDING. The assigned Agent receives an in-app notification and email alert with the endorsement details.
42. Agent reviews the endorsement JSON data. For PlanUpgrade requests, the Agent verifies the customer's eligibility (active policy, no overdue payments, no pending claims).
43. On Rejection: Agent provides a mandatory RejectionReason string (API enforces this — rejection without a reason returns HTTP 400). Customer is notified and may resubmit a corrected request.
44. On Approval: System executes the financial recalculation if the endorsement affects the premium (e.g., plan upgrade). New MonthlyPremium and CoverageAmount are computed using the same PremiumService.cs formula. An AuditLog entry is created: ActionType = 'EndorsementApproved'.

6.2 Plan Upgrade Financial Recalculation

When a PlanUpgrade endorsement is approved, the system recalculates the premium using the prorated day logic — the customer pays only for the remaining days of their current billing period at the new tier rate.

```
UpgradeAdjustment = (NewMonthlyPremium - OldMonthlyPremium) × (DaysRemaining ÷ 30)
```

Example: Customer on Silver (₹675/mo) upgrades to Gold (₹810/mo) on Day 15 of a 30-day billing period. Remaining days = 15. Adjustment Invoice = (₹810 − ₹675) × (15 ÷ 30) = ₹135 × 0.50 = ₹67.50 adjustment invoice generated.

Endorsement Type	Premium Impact	System Action
PlanUpgrade (Silver→Gold)	Positive — higher premium	Generate adjustment invoice for prorated difference

Endorsement Type	Premium Impact	System Action
PlanUpgrade (Silver→Platinum)	Positive — higher premium	Generate adjustment invoice for prorated difference
ContactUpdate	None	Update User record; AuditLog entry only
BeneficiaryChange	None	Update Beneficiary record; AuditLog entry only

6.3 Agent Commission Re-evaluation

45. Endorsement approved and premium recalculation completed.
46. If the plan tier changed, the new TierMultiplier is applied to all future MonthlyPremium calculations.
47. Agent commission for the next payment cycle is recalculated: NewMonthlyPremium × 5% = new commission amount.
48. Past commission records in AgentCommissionLog are NOT retroactively changed — only future payment events use the updated premium.

ℹ Immutability Rule: Commission records are append-only. Past calculations are preserved as a permanent financial audit trail regardless of subsequent policy changes.

7. Workflow 6 — Claim Adjudication & Coverage Deduction

Field	Detail
Goal	Process medical payout requests against individual coverage limits with strict concurrency control and atomic database deduction.
Actors	Customer, Claims Officer, System
Trigger	Active policyholder submits a claim with supporting medical documents
Concurrency Control	Exclusive lock (Take/Release) prevents multiple officers processing the same claim simultaneously
Output	Approved claim with QuestPDF Settlement Document + atomic CoverageAmount deduction, OR Rejected with mandatory reason

7.1 Hard Gates — Pre-Submission Validation

⚠ All three gates are checked BEFORE a claim enters the system. Any single failure blocks the submission entirely and returns an HTTP 400 with a descriptive error message.

Gate	Check	Block Condition	Error Returned
Gate 1	Policy Status	Claim BLOCKED if Policy.IsPaid = false OR Policy.Status = 'Inactive'	'Policy is not active. Resolve outstanding payments.'
Gate 2	Member Status	Claim BLOCKED if the submitting User.IsActive = false	'Account is inactive. Contact your Agent.'
Gate 3	Coverage Sufficiency	Claim BLOCKED if ClaimAmount > Policy.CoverageAmount	'Claim amount exceeds remaining coverage balance.'

7.2 Claim Submission Flow

SUBMITTED → PENDING → IN REVIEW (LOCKED) → APPROVED / REJECTED

49. Customer navigates to their active policy on the dashboard and selects 'File a Claim'. A claim form is displayed showing the current remaining CoverageAmount.
50. Customer enters the ClaimAmount and Reason (detailed description of the medical event). Customer uploads supporting documents: hospital bills, discharge summary, prescriptions. Documents are stored in DocumentVault with DocumentType = 'MedicalBill' or 'Prescription'.
51. System executes pre-submission gate checks (Gate 1, 2, 3 above). If all gates pass, a Claim record is created with Status = 'PendingApproval'. The Claims Officer queue is notified.
52. Claims Officer reviews the claim queue and selects 'Take' on a PENDING claim. This creates a lock record binding the claim to this specific officer. Claim.Status transitions to 'InReview'. All other officers see the claim as locked.
53. The Claims Officer reviews the uploaded medical documents in the DocumentVault viewer. The officer may view, zoom, and annotate documents before making a decision.
54. If the Officer cannot complete adjudication (e.g., missing documents), they select 'Release'. The lock is removed. Claim.Status reverts to 'PendingApproval'. The claim re-enters the open queue for any officer to take.

55. Approval Decision: Officer reviews ClaimAmount vs remaining CoverageAmount and approves or rejects the claim.

7.3 Atomic Coverage Deduction (StaffController.DecideClaim)

This is the most safety-critical operation in HIMS. The coverage deduction must be ACID-compliant — if any step fails, the entire transaction rolls back, preventing an orphaned deduction state.

56. Officer confirms approval. DecideClaim() method begins an EF Core database transaction.
57. Coverage re-check: System re-reads Policy.CoverageAmount inside the transaction. IF ClaimAmount > CoverageAmount (race condition) → ABORT transaction. Return HTTP 400.
58. Set Claim.Status = 'Approved'.
59. Compute: Policy.CoverageAmount = Policy.CoverageAmount − Claim.ClaimAmount.
60. Write AuditLog entry: ActionType = 'ClaimApprovalDeduction', OldValue = previous CoverageAmount, NewValue = updated CoverageAmount, Timestamp = UTC now.
61. Call SaveChangesAsync() — commits all changes atomically. If the AuditLog write fails, the coverage deduction is rolled back. No partial state is permitted.
62. On commit: InvoiceService generates a QuestPDF Settlement Document. The PDF is stored in DocumentVault and emailed to the customer.

7.4 Concurrency Control — Take / Release

Action	Database Operation	System Behaviour
TAKE (Lock)	INSERT ClaimLock record; UPDATE Claim.Status = 'InReview'; SET Claim.OfficerId = currentOfficer	Claim removed from open queue; all other officers see 'Locked' badge
RELEASE (Unlock)	DELETE ClaimLock record; UPDATE Claim.Status = 'PendingApproval'; SET Claim.OfficerId = null	Claim returns to open queue; any officer can now take it
APPROVE	DELETE ClaimLock; deduct coverage; generate PDF; create AuditLog	Terminal state — claim closed; coverage permanently reduced
REJECT	DELETE ClaimLock; SET Claim.Status = 'Rejected'; REQUIRE RejectionReason	Terminal state — no deduction; customer notified with reason

⚠ Rejection Rule: Any transition to REJECTED status requires a mandatory RejectionReason string. The API enforces this constraint — a rejection request without a reason string returns HTTP 400 Bad Request.

7.5 Settlement PDF Composition (QuestPDF)

PDF Section	Content
Header	Claim ID, Date of Settlement, Company Name (Health Insurance Inc.)
Claimant Details	Customer Full Name, Policy Number, Plan Tier

PDF Section	Content
Claim Summary	ClaimAmount requested, SanctionedAmount approved, Rejection Reason (if applicable)
Coverage Summary	CoverageAmount before deduction, Deduction amount, Remaining CoverageAmount
Officer Block	Claims Officer name, Employee ID, Decision Timestamp
Footer	Compliance disclaimer, document reference number, page number

8. Workflow 7 — Background Automation Service

Field	Detail
Goal	Automated daily system maintenance — policy expiry checks, invoice reminders, and account status enforcement without manual intervention.
Service Name	PolicyExpiryWorker (implements IHostedService)
Frequency	Every 24 hours — triggered automatically by the .NET hosted service infrastructure
Tasks	3 sequential tasks: Expiry Check → Status Sweep → Notification Dispatch
Trigger	Automatic — no manual intervention required. Logs all actions to AuditLog with UserId = 'SYSTEM'.

Task A — Policy Expiry Check (CheckPolicyExpiriesAsync)

63. Worker scans all ACTIVE policies (IsPaid = true, Status = 'Active') and reads the ExpiryDate field.
64. Any policy where ExpiryDate <= TODAY is flagged for expiry processing.
65. For each expired policy: Status is updated to 'Expired'. The IsPaid flag is retained as true for historical accuracy.
66. An AuditLog entry is created: ActionType = 'PolicyExpired', PolicyId, PreviousStatus = 'Active', NewStatus = 'Expired', Timestamp = UTC now.
67. Customer receives an in-app and email notification: 'Your policy [PolicyNumber] has expired. Please contact your Agent to discuss renewal options.'
68. Expired policies block all new claim submissions. The ClaimController pre-submission gate checks Policy.Status ≠ 'Expired' before accepting any claim.

Task B — Premium Due Reminders (SendPremiumRemindersAsync)

69. Worker scans all ACTIVE policies and calculates the number of days since the last confirmed payment (Policy.IsPaid = true, Policy.LastPaymentDate).
70. 7-Day Reminder: If today = LastPaymentDate + 23 days (7 days before next monthly premium due), system sends a reminder email and in-app notification to the Customer.
71. 3-Day Warning: If today = LastPaymentDate + 27 days (3 days before due), system sends an escalated warning notification.
72. Overdue Flag: If today > LastPaymentDate + 30 days and no new payment recorded, Agent receives a notification: 'Customer [Name] has an overdue premium. Intervention may be required.'

Task C — Stale Quote Cleanup (CleanStaleQuotesAsync)

73. Worker scans all PremiumQuote records where IsConvertedToPolicy = 0 (PendingReview) and CreatedAt < 90 days ago.
74. Quotes older than 90 days with no activity are marked as 'Expired' (IsConvertedToPolicy = 3). They are removed from active queues but preserved in the database for audit purposes.
75. Customer receives a notification: 'Your quote [QuoteReference] has expired. Please generate a new quote to continue your application.'

ℹ Task Execution Order: Expiry Check → Reminders → Cleanup. This order ensures that a policy is fully evaluated for expiry before reminder logic runs, preventing reminder emails from being sent to just-expired policy holders.

8.1 Worker Audit Trail

Worker Task	AuditLog ActionType	UserId in Log	Entities Modified
Policy Expiry	PolicyExpired	SYSTEM	Policy.Status
7-Day Reminder	PremiumReminderSent	SYSTEM	Notification record
3-Day Warning	PremiumWarningEscalated	SYSTEM	Notification record
Overdue Flag	AgentOverdueAlert	SYSTEM	Agent notification record
Stale Quote Cleanup	QuoteExpired	SYSTEM	PremiumQuote.IsConvertedToPolicy

9. Database, Audit Trail & Soft Delete Standards

These standards apply universally across all 7 workflows. Every state-changing operation must comply with both the soft delete policy and the audit logging requirement.

9.1 Audit Logging — Every Write Operation

⚠ Every write operation (status changes, financial calculations, locks, approvals, rejections, payment processing) MUST create an immutable AuditLog entry. This is a non-negotiable compliance requirement enforced by the Application Layer.

Required AuditLog Fields

Field	Type	Description
Id	INT, PK, IDENTITY	Auto-incremented log entry identifier
UserId	INT or 'SYSTEM'	ID of the user who performed the action; 'SYSTEM' for automated worker tasks
Entity	NVARCHAR(100)	Name of the affected entity (Policy, Claim, PremiumQuote, DocumentVault, User)
ActionType	NVARCHAR(100)	Categorized action performed (see dictionary below)
OldValue	NVARCHAR(MAX)	JSON snapshot of the entity's state BEFORE the action
NewValue	NVARCHAR(MAX)	JSON snapshot of the entity's state AFTER the action
Timestamp	DATETIME2 (UTC)	Precise UTC timestamp of the action execution

Audit Action Type Dictionary

ActionType	Triggered By	Key Fields Captured
PolicyActivation	PaymentController on payment confirmation	IsPaid: false→true; CoverageAmount set
PolicyExpired	Background worker: CheckPolicyExpiriesAsync	Status: Active→Expired
DocumentVerified	Agent approves a KYC document	DocumentVault.Status: Pending→Verified
DocumentRejected	Agent rejects a KYC document	DocumentVault.Status: Pending→Rejected; RejectionCount++
DocumentGateCleared	StaffController.VerifyPolicy — all docs verified	IsConvertedToPolicy: 0→1
PolicyRejected	RejectionCount=3 OR Agent force-rejects	IsConvertedToPolicy: 0→2
ClaimSubmitted	Customer submits a claim	Claim record created; Status=PendingApproval

ActionType	Triggered By	Key Fields Captured
ClaimLockAcquired	Claims Officer takes a claim	Status: PendingApproval→InReview; OfficerId bound
ClaimLockReleased	Claims Officer releases a claim	Status: InReview→PendingApproval; OfficerId cleared
ClaimApprovalDeduction	Officer approves claim; coverage deducted	CoverageAmount: Before→After; ClaimAmount
ClaimRejected	Officer rejects claim	Status→Rejected; RejectionReason stored
CommissionCalculated	PaymentController post-activation	AgentId, PolicyId, Rate, EarnedAmount
EndorsementApproved	Agent/Admin approves endorsement	PolicyEndorsement.Status→Approved; premium updated
StaffAccountCreated	Admin creates Agent/Officer	User.Role, User.MustChangePassword=true
PasswordReset	User or Admin resets password	User.PasswordHash updated; MustChangePassword cleared

9.2 Soft Delete Policy

ℹ HIMS never uses hard deletes on any user-facing entity. IsActive = false (soft delete) is the ONLY removal mechanism. This preserves all claim history, audit trails, and coverage records.

Entity	Soft Delete Behaviour
User (Customer/Staff)	User.IsActive = false. Soft-deleted users cannot log in but all their policies, claims, and audit records are preserved.
PremiumQuote	IsConvertedToPolicy = 2 (Rejected) or = 3 (Expired). The record is retained; it is simply excluded from active queues.
DocumentVault	DocumentVault.Status = 'Rejected'. Binary data is retained for compliance. Overwritten by a new upload on re-submission.
AgentCommissionLog	Append-only — no deletes, no updates. Historical commission records are immutable by design.
AuditLog	Append-only — no deletes, no updates. The audit trail is sacrosanct and cannot be modified by any user role including Admin.

9.3 Relational Integrity Constraints

The HIMS database schema enforces strict DeleteBehavior.Restrict on all parent-child foreign key relationships via EF Core Fluent API. This prevents orphaned records and accidental data loss.

Parent Entity	Child Entity	DeleteBehavior	Practical Effect
User	PremiumQuote	Restrict	Cannot delete a User if they have any policy records

Parent Entity	Child Entity	DeleteBehavior	Practical Effect
User	Claim	Restrict	Cannot delete a User if they have any claim history
PremiumQuote	DocumentVault	Restrict	Cannot delete a Quote if documents are attached
PremiumQuote	Claim	Restrict	Cannot delete an active policy if claims exist
User	AuditLog	Restrict	Cannot delete a User if they appear in any audit entry

9.4 Invoice Naming Convention

Invoice Type	Format	Generated By
Standard Policy Invoice	INV-[Year]-[GUID]	PaymentController on policy activation
Endorsement Adjustment Invoice	INV-[Year]-[GUID]-ADJ	EndorsementService on PlanUpgrade approval
Claim Settlement Document	CLM-[Year]-[ClaimId]-SETTLE	ClaimService on claim approval

9.5 API Security Standards

Concern	Implementation
Authentication	JWT Bearer Tokens — all routes except /auth/login and /auth/register require Authorization: Bearer <token>
Authorization	[Authorize(Roles = "...")] attribute enforced at controller level; role claims encoded in JWT at login
Password Hashing	BCrypt v2 with work factor 12 — resistant to GPU brute-force attacks
Input Validation	FluentValidation on all DTOs — invalid requests rejected at the API boundary before reaching service layer
Error Responses	Global ExceptionMiddleware intercepts all unhandled exceptions — returns RFC 7807 ProblemDetails JSON, never raw stack traces
CORS	Strictly allowlisted origins only (e.g., http://localhost:4200 in dev; production domain in prod)
Concurrency	EF Core optimistic concurrency via RowVersion / Timestamp columns on financial entities

— End of Document —

HIMS Technical Workflow Guide v1.0 | © 2025 Koushik Gaddam